

COMP90042 Natural Language Processing

Practice Exam Paper

Generated from 123-review-part1-v1.pdf using the practice-exams skill

Exam Information

Format	On-campus, closed book
Allowed aid	Non-programmable calculator
Writing time	120 minutes, plus 15 minutes reading time
Total marks	120
Structure	Part A short answer, Part B method questions, Part C algorithm questions

Coverage Blueprint

Area	Marks	Main skills tested
Preprocessing and tokenisation	10	Definitions, design choices, examples
N-gram language models	20	Derivation, smoothing, interpolation
Text classification	15	Model comparison, bias-variance, feature design
POS tagging and HMMs	25	Tagging models, emissions/transitions, Viterbi
Neural networks for NLP	20	FFN/RNN/LSTM design and interpretation
Transformers and pretrained models	20	Attention, model architecture, GPT/BERT/T5
Word embeddings	10	Distributional matrices, PMI/PPMI, evaluation

Part A: Short Answer Questions

36 marks

Answer each question in 2–3 sentences. Each question is worth 3 marks.

1. Explain the difference between sentence segmentation and tokenisation. Give one example where tokenisation is not straightforward.
2. What is the difference between derivational and inflectional morphology, and why might the distinction matter for word normalisation?
3. Contrast lemmatisation and stemming. Give one advantage and one disadvantage of each.
4. In an n-gram language model, what independence assumption is being made?
5. Why is smoothing necessary in n-gram language modelling?
6. What is the purpose of interpolation in language modelling?
7. Give two examples of text classification tasks and explain what the input and output are in each.
8. Explain the bias-variance trade-off in the context of text classification.

9. What is the difference between open-class and closed-class POS categories?
10. In an HMM POS tagger, what do emission probabilities and transition probabilities represent?
11. Why are word embeddings useful compared with one-hot word representations?
12. What is the main architectural difference between GPT and BERT?

Part B: Method Questions

60 marks

Answer all questions. Show reasoning clearly.

B1. Preprocessing Pipeline Design

[12 marks]

Topic: Preprocessing and classification. Estimated time: 12 minutes. Difficulty: medium.

You are building a sentiment classifier for short social media posts. Describe a preprocessing pipeline that includes sentence segmentation, tokenisation, subword handling, word normalisation, and stop word handling. For each step, explain one design choice and one possible risk.

B2. Smoothing Methods

[16 marks]

Topic: N-gram language models. Estimated time: 16 minutes. Difficulty: hard.

Compare add- k smoothing, absolute discounting, Katz backoff, Kneser-Ney smoothing, and interpolation. Your answer should explain the core motivation of each method and identify one situation where a more sophisticated method is preferable to add- k smoothing.

B3. Classifier Choice

[16 marks]

Topic: Text classification. Estimated time: 16 minutes. Difficulty: hard.

A course wants to classify student discussion posts by topic. Compare Naive Bayes, logistic regression, SVM, kNN, and a neural network for this task. Discuss assumptions, data requirements, interpretability, and likely bias-variance behaviour.

B4. Pretrained Model Selection

[16 marks]

Topic: Transformers and pretrained models. Estimated time: 16 minutes. Difficulty: medium.

For each task below, choose GPT, BERT, or T5 and justify your choice: sentiment classification, open-ended story generation, and rewriting a sentence into a simpler form. Include one limitation or risk for each selected model.

Part C: Algorithm Questions

24 marks

Answer both questions. Show calculations or algorithm steps.

C1. N-gram Probability and Interpolation

[12 marks]

Topic: Language modelling. Estimated time: 12 minutes. Difficulty: medium.

Suppose a corpus gives the following counts:

- $\text{count}(\text{the cat}) = 8$, $\text{count}(\text{the}) = 20$
- $\text{count}(\text{cat sat}) = 3$, $\text{count}(\text{cat}) = 10$
- $\text{count}(\text{sat down}) = 1$, $\text{count}(\text{sat}) = 5$
- unigram probabilities: $P(\text{cat}) = 0.04$, $P(\text{sat}) = 0.02$, $P(\text{down}) = 0.01$

Estimate the interpolated probability of *the cat sat down* using bigram and unigram models with $\lambda_{\text{bigram}} = 0.7$ and $\lambda_{\text{unigram}} = 0.3$. Use:

$$P(w_i \mid w_{i-1}) = 0.7P_{\text{bigram}}(w_i \mid w_{i-1}) + 0.3P_{\text{unigram}}(w_i)$$

C2. One Viterbi Step

[12 marks]

Topic: HMM POS tagging. Estimated time: 12 minutes. Difficulty: medium.

Consider a tiny HMM POS tagger with tags N and V . For the sentence *fish swim*, use the following values:

- Initial probabilities: $P(N) = 0.6$, $P(V) = 0.4$
- Emissions: $P(\text{fish} \mid N) = 0.5$, $P(\text{fish} \mid V) = 0.2$, $P(\text{swim} \mid N) = 0.1$, $P(\text{swim} \mid V) = 0.6$
- Transitions: $P(N \mid N) = 0.3$, $P(V \mid N) = 0.7$, $P(N \mid V) = 0.4$, $P(V \mid V) = 0.6$

Run Viterbi for the two words and give the most likely tag sequence with its probability.

Final Answer Key

Part A

1. Segmentation splits sentences; tokenisation splits tokens. Hard cases: abbreviations, contractions, URLs, hashtags, emojis, or no-whitespace languages.
2. Inflection changes grammatical form; derivation creates a new word/category. Normalisation should not always collapse derivational forms.
3. Lemmatisation uses linguistic analysis and dictionary forms; stemming heuristically strips affixes. Lemmas are cleaner; stems are faster but noisier.
4. The next word depends only on the previous $n - 1$ words.
5. Smoothing avoids zero probabilities and reserves probability mass for unseen n-grams.
6. Interpolation combines higher- and lower-order n-gram estimates for robustness.
7. Topic classification, sentiment analysis, or native language identification: input text, output class label.
8. High bias underfits; high variance overfits sparse text features, especially with little data.
9. Open classes admit new words, e.g. nouns/verbs; closed classes are relatively fixed, e.g. determiners/prepositions.
10. Emissions: $P(\text{word} \mid \text{tag})$. Transitions: $P(\text{tag}_i \mid \text{tag}_{i-1})$.
11. Embeddings are dense, similarity-aware representations; one-hot vectors are sparse and do not encode semantic closeness.
12. GPT is decoder-only/autoregressive for generation; BERT is encoder-only/masked-LM for contextual representation.

Part B

B1. Suitable pipeline: sentence segmentation, social-media-aware tokenisation, subword handling for rare/OOV words,

cautious lemmatisation/stemming, and selective stop-word removal. Main risks: losing negation, emojis, hashtags, domain terms, or sentiment cues.

B2. Add- k : simple count adjustment but often over-smooths. Absolute discounting: subtracts fixed mass from seen n-grams. Katz: discounted seen n-grams plus backoff for unseen ones. Kneser-Ney: uses continuation probability, good for sparse data. Interpolation: mixes orders directly.

B3. NB: simple/high-bias/independence assumption. Logistic regression: discriminative and interpretable. SVM: strong for sparse high-dimensional features. kNN: simple but distance-sensitive and costly. Neural networks: richer representations, more data/regularisation, higher variance.

B4. Sentiment: BERT, because encoder representations support classification. Story generation: GPT, because decoder-only autoregression supports continuation. Simplification: T5, because encoder-decoder text-to-text modelling supports rewriting. Mention risks such as hallucination, data mismatch, or limited controllability.

Part C

C1.

$$P(c \mid t) = 0.7(8/20) + 0.3(0.04) = 0.292$$

$$P(s \mid c) = 0.7(3/10) + 0.3(0.02) = 0.216$$

$$P(d \mid s) = 0.7(1/5) + 0.3(0.01) = 0.143$$

Final probability, ignoring the first-word prior:

$$0.292 \times 0.216 \times 0.143 = 0.009018816.$$

C2. Initialise: $N = 0.6 \times 0.5 = 0.30$, $V = 0.4 \times 0.2 = 0.08$. For *swim*: ending $N = \max(0.009, 0.0032) = 0.009$; ending $V = \max(0.126, 0.0288) = 0.126$. Best sequence: N, V , probability 0.126.